UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
DEPARTMENT OF COMPUTER AND SOFTWARE ENGINEERING
ACADEMIC YEAR: 2025- 2026
YEAR OF STUDY: YEAR III
MODULE: MOBILE APPLICATION SYSTEMS AND DESIGN
DATE: On Monday, March 16th, 2026
REG. NUMBER: 223001019

FLUTTER LAB 4: CONSUMING API IN FLUTTER

Question 1: Dependencies used and why.

* http: ^1.2.0
The "http" package is flutter's standard library for making HTTP requests. It provides simple functions like http.get(), http.post(), http.put, and http.delete() which directly map to the REST operations needed for the posts manager (CRUD). It is lightweight, officially supported by the Dart team, and integrates well with Dart's sync/await model via Flutter objects.

* google_fonts: ^6.1.0:
The "google_fonts" package allows access to Google Fonts directly in Flutter without manually bundling font-files. It was used to apply the SpaceGrotesk display font and Inter body font, giving the app a modern, polished aesthetic. This is purely a UI/UX dependency, not required for API integration.

* built-in dart: convert:
The "dart: convert" library (built into Dart, no pubspec entry needed) provides jsonDecode() and jsonEncode(). These are used to deserialize JSON responses from the API into Post objects, and to serialize post objects into JSON request bodies.

Question 2: Handling API exceptions.

In Flutter, robust API exception handling involves multiple layers.

Exception Type	Cause	How we handle it
SocketException	No internet/DNS failure.	Catch and throw ApiException('No internet Connection').
HttpException	Server unreachable or protocol error.	Catch and rethrow as ApiException.

FormatException	Response body is not valid JSON	catch and throw ApiException('Invalid data')
TimeoutException	Request takes too long (>15sec)	set.timeout(Duration(seconds:15)) on request
Non-200 status code.	Server returned 404, 500, etc.	Check response.statusCode and throw ApiException.

General pattern used in PostService:
A Custom ApiException class was created that wraps all errors with a human-readable message and optional HTTP status code. Every service method uses a try/catch block. In the UI, the FutureBuilder's error branch and Snackbar messages display these user-friendly messages instead of raw exception details.

Question 3: The FutureBuilder widget

- What is a Future?

A Future<T> in Dart represents a value that is not yet available but will be provided at some point in the future - analogous to a Promise in JavaScript. API calls like http.get() return Futures because the network ~~access~~ response takes time to arrive.

- What is FutureBuilder?

The FutureBuilder widget rebuilds its subtree whenever the state of a Future changes. It accepts a future and a builder call back which receives an AsyncSnapshot. the snapshot's connectionState property indicates whether the future is still loading, completed with data, or completed with an error.

- How it is used in Posts Manager?

In PostListScreen, the entire list body is wrapped in a FutureBuilder that depends on _postsFuture (a Future<List<Post>> returned by PostService.fetchPosts()). The UI responds to three states:

connectionState	widget shown	description
waiting	CircularProgressIndicator.	spinner shown while HTTP request is in-flight.
done (hasError)	Error card + Retry button.	Network/parse error shown with user-friendly message.
done (hasData)	ListView.builder with PostCard.	All posts rendered as tappable cards.

The _postsFuture variable is reset by calling setState() and reassigning it whenever a post is created, updated, or deleted, this forces FutureBuilder to refetch and redisplay the updated list.

END